

# Analisis tecnico del bug de desalineacion semantica en PCZT

Documento interno de auditoria tecnica

7 de mayo de 2026

## Índice

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>1. Resumen ejecutivo</b>                                     | <b>2</b> |
| <b>2. Contexto</b>                                              | <b>2</b> |
| 2.1. Que es PCZT? . . . . .                                     | 2        |
| 2.2. Hipotesis inicial . . . . .                                | 3        |
| <b>3. Preguntas que guiaron la investigacion</b>                | <b>3</b> |
| <b>4. Mapa de fuentes de verdad</b>                             | <b>4</b> |
| 4.1. Fuente de verdad criptografica . . . . .                   | 4        |
| 4.2. Fuente de verdad de presentacion . . . . .                 | 4        |
| <b>5. Trazado del flujo vulnerable</b>                          | <b>4</b> |
| 5.1. Creacion del PCZT . . . . .                                | 4        |
| 5.2. Firma . . . . .                                            | 5        |
| 5.3. Extraccion y almacenamiento . . . . .                      | 5        |
| <b>6. Hipotesis exacta del exploit</b>                          | <b>5</b> |
| <b>7. PoC ejecutable</b>                                        | <b>6</b> |
| 7.1. Test agregado . . . . .                                    | 6        |
| 7.2. Que hace el test . . . . .                                 | 6        |
| 7.3. Comando de ejecucion . . . . .                             | 6        |
| 7.4. Salida observada . . . . .                                 | 7        |
| 7.5. Resultado semantico exacto . . . . .                       | 7        |
| 7.6. Interpretacion . . . . .                                   | 7        |
| <b>8. Por que las firmas siguen siendo validas</b>              | <b>7</b> |
| <b>9. Que parte del sistema consume la metadata falsificada</b> | <b>8</b> |
| <b>10. Preguntas que nos hicimos hasta llegar al PoC</b>        | <b>8</b> |
| <b>11. Que no demostramos todavia</b>                           | <b>8</b> |
| 11.1. Signer deception real frente a una UI humana . . . . .    | 8        |
| 11.2. Orchard . . . . .                                         | 8        |
| <b>12. Causa raiz</b>                                           | <b>9</b> |
| <b>13. Clasificacion tecnica</b>                                | <b>9</b> |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>14. Severidad e interpretacion</b>                          | <b>9</b>  |
| 14.1. Lo que si es . . . . .                                   | 9         |
| 14.2. Lo que no es . . . . .                                   | 9         |
| <b>15. Recomendaciones de remediacion</b>                      | <b>9</b>  |
| <b>16. Respuesta final</b>                                     | <b>10</b> |
| 16.1. Is the mismatch fully demonstrated end-to-end? . . . . . | 10        |

# 1. Resumen ejecutivo

Este documento describe, de punta a punta, el proceso que seguimos para encontrar, validar y convertir en PoC un bug semantico en el subsistema PCZT (*Partially Constructed Zcash Transaction*) dentro de `librustzcash`.

La conclusion central es la siguiente:

Es posible construir un PCZT valido cuyo output shielded comprometido paga al recipient B, pero cuya metadata auxiliar hace que la capa de reconstruccion de historial/salidas enviadas registre y exponga al recipient A.

En otras palabras:

- **Recipient comprometido criptograficamente: B**
- **Recipient mostrado/almacenado en sent history: A**
- **La transaccion sigue siendo valida y se acepta: si**

Lo que **si** demostramos:

- un flujo end-to-end dentro de `zcash_client_backend` donde:
  - se crea un PCZT valido,
  - se muta solo metadata de presentacion,
  - se firman los spends,
  - se extrae la transaccion final,
  - y la capa de almacenamiento de salidas enviadas persiste un recipient diferente del comprometido.

Lo que **no** demostramos todavia:

- que una UI real de confirmacion de firma muestre A al usuario humano antes de firmar;
- que el mismo PoC este repetido para Orchard;
- que un wallet de produccion externo a este repo renderice esta metadata como fuente autoritativa en su pantalla de aprobacion.

## 2. Contexto

### 2.1. Que es PCZT?

PCZT es el formato de transaccion parcialmente construida de Zcash. La idea es separar la construccion de una transaccion en roles:

- `Creator`
- `Updater`
- `IoFinalizer`
- `Prover`
- `Signer`

- Combiner
- Redactor
- SpendFinalizer
- TransactionExtractor
- Verifier

Cada rol agrega, valida, poda o transforma informacion.

## 2.2. Hipotesis inicial

La hipotesis que queriamos probar era:

Un coordinador malicioso puede hacer que un participante vea semanticamente “recipient A” mientras la salida realmente comprometida por la transaccion paga a “recipient B”.

La forma mas concreta de esto en el codigo era:

- recipient comprometido en campos shielded del output;
- recipient de presentacion en campos auxiliares como:
  - `user_address`
  - metadata propietaria `PROPRIETARY_OUTPUT_INFO`
  - `PcztRecipient::External`

## 3. Preguntas que guiaron la investigacion

Estas fueron las preguntas tecnicas que nos fuimos haciendo durante la busqueda:

1. Cuales campos estan comprometidos criptograficamente y cuales son solo metadata?
2. El recipient que ve un signer o un consumidor downstream sale del output real o de metadata?
3. Hay algun chequeo generico que garantice:

`user_address = recipient comprometido`

antes de firmar?

4. El **Signer** de alto nivel valida la semantica de outputs o solo la validez minima para firmar?
5. `extract_and_store_transaction_from_pczt` reconstruye el recipient desde el note o desde metadata?
6. Si mutamos solo `user_address`, las firmas siguen siendo validas?
7. Si las firmas siguen siendo validas, el historial guardado usa A o usa B?

## 4. Mapa de fuentes de verdad

### 4.1. Fuente de verdad criptografica

Para outputs shielded, el recipient comprometido depende de:

- en Sapling:
  - `recipient`
  - `value`
  - `rseed`
  - `ciphertext`
  - `commitment`
- en Orchard:
  - `recipient`
  - `value`
  - `rho`
  - `rseed`
  - `ciphertext`
  - `commitment`

Esos campos alimentan:

- el note comprometido,
- la recuperacion del output,
- la semantica on-chain real.

### 4.2. Fuente de verdad de presentacion

Para recipients externos, en el flujo PCZT aparecen dos piezas auxiliares:

- `user_address`
- `PROPRIETARY_OUTPUT_INFO` con `PcztRecipient::External`

Estas piezas **no** estan comprometidas en sighash ni en commitments.

## 5. Trazado del flujo vulnerable

### 5.1. Creacion del PCZT

En `zcash_client_backend/src/data_api/wallet.rs`, el flujo `create_pczt_from_proposal` hace dos cosas importantes:

1. toma el recipient semantico de la propuesta;
2. lo traduce a metadata auxiliar:

- `user_address`
- `PROPRIETARY_OUTPUT_INFO`

Para outputs externos, guarda:

`PcztRecipient::External`

y tambien serializa la direccion de usuario.

## 5.2. Firma

El Signer de alto nivel:

- calcula el `shielded_sighash` desde `TransactionData<EffectsOnly>`;
- firma los spends;
- no impone un chequeo output-side de consistencia entre:
  - recipient comprometido,
  - y `user_address`.

Esto es crucial: si mutamos `user_address`, las firmas no cambian.

## 5.3. Extraccion y almacenamiento

En `extract_and_store_transaction_from_pczt` ocurre el punto clave.

La funcion:

- reconstruye el note desde los campos comprometidos;
- pero reconstruye el recipient externo para historial usando metadata:
  - `PcztRecipient::External`
  - `user_address`

En el helper `to_sent_transaction_output(...)`:

```
(PcztRecipient::External, Some(addr)) =>
  Recipient::External { recipient_address: addr, ... }
```

O sea: el recipient externo almacenado sale de metadata, no del recipient comprometido del note.

## 6. Hipotesis exacta del exploit

La condicion necesaria y suficiente es:

1. construir un PCZT valido que pague a B;
2. mutar solo `user_address = A`;
3. dejar:
  - recipient comprometido = B
  - ciphertext / note / commitment = intactos
4. firmar normalmente;
5. extraer/almacenar;
6. observar si el recipient almacenado es A.

## 7. PoC ejecutable

### 7.1. Test agregado

Se agrego el siguiente test:

```
data_api::wallet::tests::  
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling
```

Archivo:

```
zcash_client_backend/src/data_api/wallet.rs
```

### 7.2. Que hace el test

El test hace exactamente esto:

1. crea una transaccion valida transparent ->Sapling;

2. recipient comprometido real:

```
committed_recipient_addr = B
```

3. recipient falso de presentacion:

```
fake_displayed_addr = A
```

4. construye el PCZT;

5. muta solo:

```
output.user_address = A
```

6. preserva:

```
PcztRecipient::External
```

7. crea proofs;

8. firma;

9. finaliza spends transparentes;

10. extrae la transaccion final;

11. recupera el recipient comprometido via OVK del sender;

12. persiste el resultado en un MockWalletDb instrumentado para capturar SentTransactionOutput;

13. compara el recipient recuperado con el recipient almacenado.

### 7.3. Comando de ejecucion

```
cargo test -p zcash_client_backend --features pczt \  
  extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling \  
  \  
  -- --nocapture
```

## 7.4. Salida observada

```
running 1 test
test data_api::wallet::tests::
extract_and_store_transaction_from_pczt_can_store_metadata_recipient_distinct_from_committed_sapling
... ok
```

## 7.5. Resultado semantico exacto

El test prueba estas igualdades/desigualdades:

- recipient comprometido recuperado del tx:

```
recovered_recipient_addr == committed_recipient_addr == B
```

- recipient mostrado/almacenado:

```
displayed_recipient == fake_displayed_addr == A
```

- mismatch:

```
displayed_recipient != recovered_recipient_addr
```

## 7.6. Interpretacion

Esto equivale a:

Committed recipient: B

Displayed/stored recipient: A

Transaction accepted: yes

## 8. Por que las firmas siguen siendo validas

Porque el campo mutado, `user_address`, es metadata auxiliar.

No entra en:

- el `shielded_sighash`,
- el recipient comprometido del note,
- el ciphertext,
- el commitment.

Entonces:

- el recipient real B queda criptograficamente intacto;
- la metadata A sigue viva para presentacion/almacenamiento;
- la verificacion criptografica no detecta el desalineamiento.

## 9. Que parte del sistema consume la metadata falsificada

La capa vulnerable no es la verificación de la transacción on-chain.

La capa vulnerable es la reconstrucción de `SentTransactionOutput`:

- `Recipient::External` se construye desde metadata,
- el note/memo viene del output comprometido,
- el historial resultante mezcla ambas fuentes.

Eso convierte a la inconsistencia en una vulnerabilidad de integridad semántica del historial/sumario.

## 10. Preguntas que nos hicimos hasta llegar al PoC

Estas fueron las preguntas más útiles que guiaron el hallazgo:

1. El recipient que ve el consumidor downstream sale del output real o de metadata?
2. `user_address` está comprometido en sighash? No.
3. Existe algún chequeo genérico:

`user_address = recipient comprometido`

antes de firmar? No.

4. `extract_and_store_transaction_from_pczt` usa metadata para clasificar recipients externos? Si.
5. Entonces, si mutamos solo `user_address`, se rompe la firma? No.
6. Si no se rompe la firma, se persiste A o B? Se persiste A.

## 11. Que no demostramos todavía

### 11.1. Signer deception real frente a una UI humana

No demostramos todavía que exista, dentro de este repo, una UI de confirmación de firma que muestre `user_address = A` al usuario humano y le haga aprobar mientras el output real paga a B.

Por eso la evaluación final correcta es:

- **Sent-history deception:** sí, demostrado
- **Signer-confirmation deception:** incierto en este codebase

### 11.2. Orchard

No repetimos aún el mismo PoC completo para Orchard.

## 12. Causa raiz

La causa raiz no es una bug criptografica clasica.  
Es una **fractura entre dos fuentes de verdad**:

- fuente criptografica del recipient real;
- fuente auxiliar de presentacion del recipient externo.

Y esa fractura queda habilitada por tres propiedades simultaneas:

1. la metadata de recipient externo es mutable;
2. esa metadata no esta comprometida en la firma;
3. la capa de reconstruccion/almacenado la usa como autoritativa.

## 13. Clasificacion tecnica

La mejor forma de describir el bug es:

**semantic binding failure between committed shielded recipient and stored/displayed external recipient metadata in PCZT-derived sent-history flows**

## 14. Severidad e interpretacion

### 14.1. Lo que si es

- bug de integridad semantica;
- coordinador malicioso puede sembrar metadata enganosa;
- el sistema puede almacenar y presentar esa metadata como recipient externo.

### 14.2. Lo que no es

- no es forgery;
- no es bypass de ownership de spend;
- no es funds theft demostrada por si sola;
- no es inconsistencia de consenso.

## 15. Recomendaciones de remediacion

1. No usar **user\_address** como fuente autoritativa de recipient externo si no fue verificada contra el recipient comprometido.
2. Recomputar recipient de salida desde campos comprometidos siempre que sea posible.
3. En el signer de alto nivel, agregar validacion explicita de coherencia output-side.
4. Si se conserva metadata de presentacion, marcarla explicitamente como advisory y no mezclarla con objetos de “recipient real”.
5. Agregar el mismo PoC para Orchard.

## 16. Respuesta final

### 16.1. Is the mismatch fully demonstrated end-to-end?

**PARTIALLY**

Explicacion exacta:

- **Si**, esta completamente demostrado end-to-end para:
  - construccion del PCZT,
  - mutacion de metadata,
  - firma,
  - extraccion,
  - almacenamiento de recipient mostrado/guardado.
- **No**, no esta completamente demostrado para:
  - una pantalla real de confirmacion de firma mostrada a un humano.

La respuesta operacional mas precisa es:

El mismatch esta **plenamente demostrado** como bug de sent-history / stored-recipient deception dentro del flujo PCZT de librustzcash.

Todavia esta **sin demostrar** como bug de signer-confirmation deception en una UI real.